UTM
UNIVERSITI TEKNOLOGI MALAYSIA

Chapter 1: Introduction to Object-Oriented Programming & Java (Part 3)

SECJ2154: Object-Oriented Programming
Faculty of Computing (FC), UTM

Note: This slide is based on Dr. Norazah Yusof's slide
Updated by Shahida Sulaiman, 2025
Other sources are cited accordingly
Reminder: Do not upload this slide to external platforms

Innovating Solutions

UTM JOHOR BAHRU

Contents

- **Creating Java Program:**
  - Identifiers
  - Variables
- **Assignment Statement**
- **Classes in Java:**
  - String
  - Scanner
  - JOption Pane Class

Identifiers

JAVA PROGRAM

- All Java components (classes, variables, and methods) need **names**
- In Java, these names are called **identifiers**
- There are rules for what constitutes a **legal Java identifier**
- Besides legal identifiers, Java programmers (and Sun) have created **conventions** for naming methods, variables, and classes.

Rules for Naming Identifiers

JAVA PROGRAM

- Identifiers **must start** with a letter, a currency character ($) or a connecting character e.g. the underscore (_) but **cannot start with a number or any other special characters**
- The following can contain any combination of letters, currency characters, connecting characters, or numbers.
- Identifiers **can be any length**, there is no limit to the number of characters an identifier can contain.
- Cannot use a Java keyword as an identifier
- Identifiers in Java are **case-sensitive** e.g. utm and Utm are two different identifiers.
- Identifiers cannot be true, false or null.

JAVA PROGRAM

Legal/illegal identifier?

1. int _a;
2. int $c;
3. int 2_w;
4. int .f;
5. int -d;
6. int _$;
7. int this_is_a_detailed_name_for_identifier;
8. int :b;
9. int e#;
10. int 7g;

Programming Style and Documentation

JAVA PROGRAM

- Naming Conventions:
  - Classes and Interfaces
  - Methods
  - Variables and Constants
- Appropriate Comments
- Proper Indentation and Spacing Lines
- Block Styles

Sun's Java Code Conventions: Classes and Interfaces

JAVA PROGRAM

The naming standards that Sun recommends:

- **Classes and interfaces:** The **first letter should be capitalized**, and if several words are linked together to form the name, the first letter of the inner words should be uppercase (a format known as "`CamelCase`" resembles the humps of a camel).
- For **classes**, the names should typically be **nouns** e.g.
  - Account
  - Customer
  - OrderHistory
- For **interfaces**, the names should typically be **adjectives** e.g.
  - Runnable
  - Serializable

Sun's Java Code Conventions: Methods

JAVA PROGRAM

- **Methods:** The **first letter should be lowercase**, and then normal camelCase rules should be used
- In addition, the names should typically be **verb-noun pairs** e.g.
  - `getBalance`
  - `doCalculation`
  - `setCustomerName`
  - `calcArea`

Sun's Java Code Conventions: Variables and Constants

JAVA PROGRAM

- **Variables:** Like methods, the camelCase format should be used, **starting with a lowercase letter**.
- Sun recommends short, meaningful names e.g.
  - `nama`
  - `buttonWidth`
  - `accountBalance`
  - `myString`
- **Constants:** Java constants are created by marking variables static and final.
- They should be named using **uppercase letters** with underscore characters as separators e.g. `MIN_HEIGHT`

JAVA PROGRAM

Appropriate Comments

- Include a **summary** at the beginning of the program to explain what the program does, its key features, its supporting data structures, and any unique techniques it uses.
- **For this course**: Include your name, class section, lecturer, date, and a brief description at the beginning of the program.

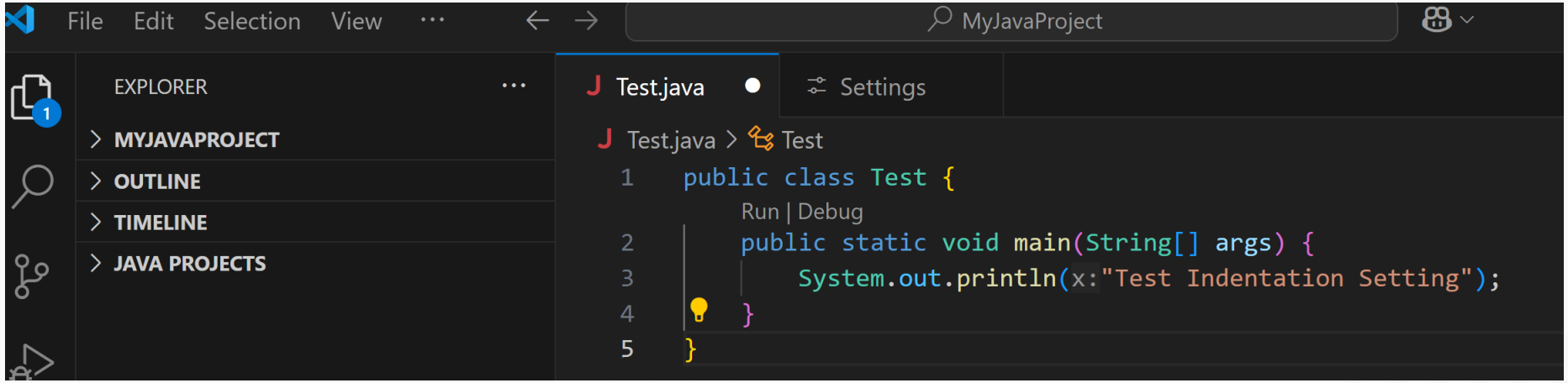
```java
/*
 * Name        : [Your Name]
 * Section     : [Your Section]
 * Lecturer    : [Lecturer's Name]
 * Date        : [Submission Date]
 */

public class StudentSubmission {
    public static void main(String[] args) {
        // Print a simple message to demonstrate functionality
        System.out.println("This is my Java program for submission!")
    }
}
```

JAVA PROGRAM

Proper Indentation and Spacing

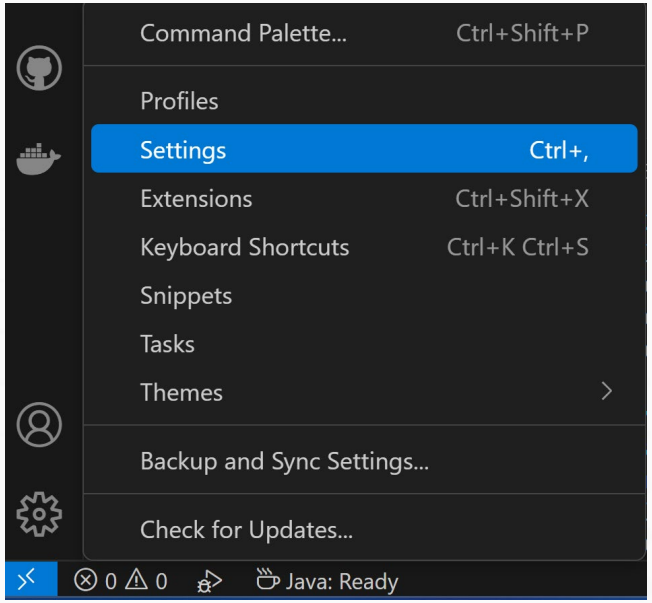
- Indentation: Indent two spaces; industry practice mostly set tab as **four spaces** (https://code.visualstudio.com/docs/editing/codebasics)
- Spacing: Use blank lines to separate segments of the code
- See e.g. Test.java program via Visual Studio Code

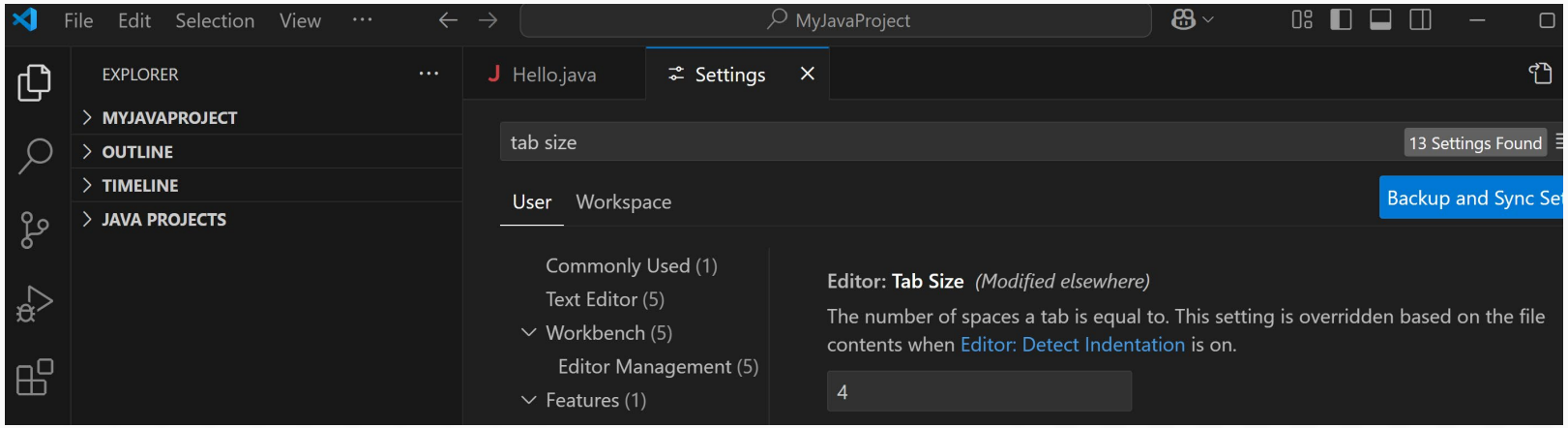


Set the Tab Size for Indentation in Visual Studio Code

JAVA PROGRAM

- Click on the Gear icon (bottom left), select Settings
- Search “tab size” to view the set size (default is usually 4 spaces)





Block Styles

JAVA PROGRAM

- Use end-of-line style for braces or next-line style
- Be consistent in one program or one project

Next-line style

```
public class Test
{
  public static void main(String[] args)
  {
    System.out.println("Block Styles");
  }
}
```

End-of-line style

```
public class Test {
  public static void main(String[] args) {
    System.out.println("Block Styles");
  }
}
```

Variables

JAVA PROGRAM

- Variables are used to store data in a program
- There are **two types** of variables in Java:
  1. **Primitives**
  2. **Reference variables** that can be declared as:
     - static variables
     - instance variables
     - method parameters, or
     - local variables

JAVA PROGRAM

Primitives Variables

- A primitive can be one of **eight types**: char, boolean, byte, short, int, long, double, or float.
- Once a primitive has been declared, its **primitive type can never change**, although in most cases its value can change.
- Primitive variables **can be declared as** class variables (statics), instance variables, method parameters, or local variables.
- One or more primitives of the same primitive type, can be declared in a single line.
- Examples of primitive variable declarations:
  - `char huruf;`
  - `boolean myBooleanPrimitive;`
  - `int x, y, z;  // declare three int primitives`
  - `double area;`

Declaring primitive type

JAVA PROGRAM

- Example of **declaring** Java variables with primitive types:

```
int numberOfBeans;
double oneWeight, totalWeight;
```

- The **declaration** of a variable **can be combined** with its initialization via an assignment statement

```
int count = 0;
double distance = 55 * .5;
char grade = 'A';
```

- Note that some variables **can be initialized** and others can remain uninitialized in the same declaration

```
int initialCount = 50, finalCount;
```

JAVA PROGRAM

Display 1.2 Primitive Types

TYPE NAME	KIND OF VALUE	MEMORY USED	SIZE RANGE
`boolean`	`true` or `false`	1 byte	not applicable
`char`	single character (Unicode)	2 bytes	all Unicode characters
`byte`	integer	1 byte	−128 to 127
`short`	integer	2 bytes	−32768 to 32767
`int`	integer	4 bytes	−2147483648 to 2147483647
`long`	integer	8 bytes	−9223372036854775808 to 9223372036854775807
`float`	floating-point number	4 bytes	$-3.40282347 \times 10^{+38}$ to $-1.40239846 \times 10^{-45}$
`double`	floating-point number	8 bytes	$\pm 1.76769313486231570 \times 10^{+308}$ to $\pm 4.94065645841246544 \times 10^{-324}$

JAVA PROGRAM

Reference Variables

- A **reference variable** is used to **refer to (or access) an object**, it is declared to be of a specific type and that type can never be changed.
- It can be used to **refer to any object of the declared type**, or of a **subtype** of the declared type (a compatible type).
- Reference variables can be declared as **static variables**, **instance variables**, **method parameters**, or **local variables**.
- One or more reference variables, of the same type, can be declared in a single line.
- Examples of reference variable declarations:
  - `Object o;`
  - `Animal myNewPetReferenceVariable;`
  - `String s1, s2, s3; //declare three String vars.`

Reference Variables: Declared as Instance Variables

JAVA PROGRAM

- **Instance variables** are defined **inside the class, but outside of any method**, and are only initialized when the class is instantiated; the fields that belong to each unique object.
- E.g. the following code defines **fields** (instance variables) for the radius of circle objects:

```
public class Circle extends Object { //Class header
    private double radius;
}
```

- Each circle instance will know its own radius. In other words, each instance can have its own unique values for this field.
- The term "field," "instance variable," "property," or "attribute," mean virtually the same thing.

Reference Variables: Declared as Local Variables

JAVA PROGRAM

- **Local variables** are variables **declared within a method** i.e. start their life inside the method and are also destroyed when the method has completed.
- Local variables are always **on the stack**, not the heap.
- Although the value of the variable might be passed into, say, another method that then stores the value in an instance variable, the variable itself **lives only within the scope of the method**.
- Before a local variable can be *used*, it **must be initialized** with a value e.g.

```
class TestServer {
     public void logIn() {
       int count = 10;
     }
}
```

Local Variables

JAVA PROGRAM

- Local variable declarations **cannot use most of the modifiers** that can be applied to instance variables (public, private, protected), transient, volatile, abstract, or static; but can be marked **final**.
- A local variable **cannot be referenced in any code outside** the method in which it is declared e.g. below.
- It would be impossible to refer to the variable count anywhere else in the class except within the scope of the method `logIn()`.
- However, the **value of count can be passed out of the method** to take on a new life.

```
class TestServer {
  public void logIn() {
    int count = 10;
  }
  public void doSomething(int i) {
    count = i; // Won't compile! Can't access
               // count outside method login()
  }
}
```

Local Variables

JAVA PROGRAM

- It is possible to declare a local variable with the **same name** as an instance variable.
- It is known as **shadowing** (see e.g.)
- The code produces the following output:

```
public class TestServer {
  int count = 9; //An instance variable named count
  public void logIn(){
    int count = 10; // A local variable named count
    System.out.println("local variable count is "
       + count);
  }
  public void count () {
    System.out.println("instance variable count is "
    + count);
  }
  public static void main(String[] args) {
    new TestServer().logIn();
    new TestServer().count();
  }
 }
```

```
local variable count is 10
instance variable count is 9
```

JAVA PROGRAM

Final Variables

- Declaring a variable with the **final keyword** makes it a **constant:** represent permanent data that never changes.
- For primitives, this means that once the variable is assigned a value, the value cannot be altered.
- Syntax to declare a constant:
  - `final datatype CONSTANTNAME = VALUE;`
- Example:
  - `final double PI = 3.14159; //PI as constant`
  - `final int SIZE = 3;`

Final Variables (Is it legal?)

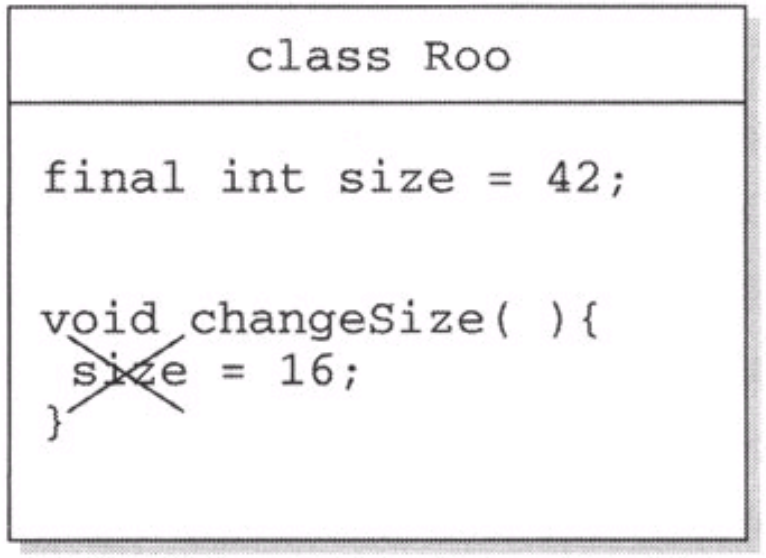
JAVA PROGRAM

```
class Roo {
  final int size = 42;

  public void changeSize() {
    size = 16;
  }
}
```

Note: size should be written as uppercase SIZE as it is final

final variable



final *variable* cannot be assigned a new value, once the initial method is made (the initial assignment of a value must happen before the constructor completes).

Final Variables

JAVA PROGRAM

- A reference variable marked final **cannot be reassigned** to refer to a different object.
- The **data within the object can be modified**, but the reference variable cannot be changed.
- In other words, a final reference still allows us to **modify the state of the object** it refers to, but we **cannot modify the reference variable** to make it refer to a different object.

Variable Scoping

JAVA PROGRAM

Once a variable is declared and initialized, a natural question is "How long will this variable be around?"

```
public class Layout { // class
  static int s = 343; // static variable
  int x; // instance variable
  {  x = 7;
     int x2 = 5; } // initialization block

  public Layout() { // constructor
    x += 8;
    int x3 = 6;
  }

  public void doStuff() { // method
    int y = 0; // local variable
    for(int z = 0; z < 4; z++) { // 'for' code block
      y += z + x;
    }
  }
}
```

- `s` is a static variable
- `x` is an instance variable
- `y` is a local variable (sometimes called a "method local" variable)
- `z` is a block variable
- `x2` is an init block variable, a flavor of local variable
- `x3` is a constructor variable, a flavor of local variable

Variable Scoping

JAVA PROGRAM

There are **four basic scopes**:

1. **Static variables** have the longest scope; they are created when the class is loaded, and they survive as long as the class stays loaded in the JVM.
2. **Instance variables** are the next most long-lived; they are created when a new instance is created, and they live until the instance is removed.
3. **Local variables** are next; they live as long as their method remains on the stack. However, local variables can be alive, and still be "out of scope".
4. **Block variables** live only as long as the code block is executing.

Scoping Errors

JAVA PROGRAM

One **common mistake** happens when a variable is shadowed and two scopes overlap:

- Attempting to access an **instance variable** from a static context, typically from main()
- Attempting to access a **local variable** from a nested method
- Attempting to use a **block variable** after the code block has completed

```
class ScopeErrors1 {
  int x = 5;
  public static void main(String[] args) {
   x++;   // won't compile, x is an 'instance' variable
  }
}
```

```
public class ScopeErrors2 {
   public static void main(String [] args) {
     ScopeErrors s = new ScopeErrors();
     s.go() ;  }

   void go() {
     int y = 5;
     go2() ;
     y++;   // once go2() completes, y is back in scope
   }

   void go2() {
     y++;          // won't compile, y is local to go()
   }
}
```

```
void go3() {
 for(int z = 0; z < 5; z++) {
 boolean test = false;
 if(z == 3) {      test = true;      break; }
 }  System.out.print(test); // 'test' is an ex-variable,
                            // it has ceased to be...}
```

ASSG. STATEMENT

Assignment Statement

- In Java, the assignment statement is used to **change the value** of a variable using the **equal sign** (=) as the assignment operator
- An assignment statement consists of a variable on the left side of the operator, and an ***expression* on the right side** of the operator:

```
Variable = Expression;
```

  - An *expression* consists of a variable, number, or mix of variables, numbers, operators, and/or method invocations e.g.

```
temperature = 98.6;
count = numberOfBeans;
```

- The assignment operator is automatically **executed from right-to-left**, so assignment statements can be chained e.g.

```
number2 = number1 = 3;
```

Assignment Compatibility

ASSG. STATEMENT

- In general, the value of **one type cannot be stored** in a variable of another type.
- A `double` value cannot be stored in an `int` variable e.g.

```
int intVariable = 2.99; //Illegal
```

- An `int` cannot be assigned to a variable of type `boolean`, nor can a `boolean` be assigned to a variable of type int
- However, there are exceptions to this: An `int` value can be stored in a `double` type e.g.

```
double doubleVariable = 2;
```

Assignment Compatibility: Example

ASSG. STATEMENT

- What would happen if we assign an integer literal to a byte variable?

```
byte b = 3;
byte c = 8;
byte d = 500;
```

- Can assign an integer value to a byte type variable, but the value must fall within the range of the byte type.
- In most programming languages, a byte typically holds values from **-128 to 127** (for signed bytes) or **0 to 255** (for unsigned bytes), Java can represent values for **signed bytes** only

```
error: incompatible types: possible lossy conversion from int to byte
    byte d = 500;
             ^
```

ASSG. STATEMENT

Type Casting

- Casting converts primitive values from one type to another.
- Two type of casting:
  1. Implicit casting – the conversion happens automatically
  2. Explicit casting – programmer tells the compiler the type to cast
- Example of **implicit cast**:

```
int a = 100;
long b = a; // Implicit cast, an int value always
            // fits in a long
```

- Example of **explicit cast**:

```
double d = 4.5;
int i;
i = (int)d; // The value of variable i is 4
            // The value of variable d is 4.5
            // Explicit cast, the integer could lose info
```

Widening Conversion

- Putting a smaller thing (say, a byte) into a bigger container (like an int)
- An **implicit cast** happens when doing a widening conversion:
  - small-value-into-large-container
- Integer values may be assigned to a double variable without explicit casting, because any integer value can fit in a 64-bit double e.g.

```
double d = 100L; // Implicit cast
```

Narrowing Conversion

- Putting a larger thing (say, a `long`) into a smaller container (like an `int`)
- The large-value-into-small-container conversion requires **explicit cast**
- An explicit type cast is required to assign a value of larger type (e.g., `float`) to a variable with smaller value type (e.g. `int`)
  - Example:

```
float a = 100.001f;
int b = (int)a; // Explicit cast, the float
                // could lose info
```

Assignment Compatibility

- More generally, a value of any type in the following list **can be assigned** to a variable of any type that **appears to the right** of it

```
byte → short → int → long → float → double
char ————————————↑
```

- Note that as we move down the list from left to right, **the range** of allowed values for the types **becomes larger**

SELF-TEST

Exercise

- Are the following assignment statements legal/illegal?

```
1. int x = 3957.229;
2. long s = 56L; byte b = (byte)s;
3. byte a = 128;
```

Arithmetic Operators and Expressions

- As in most languages, expressions can be formed in Java using variables, constants, and arithmetic operators.
- Java has **FIVE (5) arithmetic operators:**

Operator	Meaning	Type	Example
+	Addition	Binary	total = cost + tax;
-	Subtraction	Binary	cost = total – tax;
*	Multiplication	Binary	tax = cost * rate;
/	Division	Binary	salePrice = original / 2;
%	Modulus	Binary	remainder = value % 5;

Shorthand Assignment Statement

ASSG. STATEMENT

- Shorthand assignment notation combines the assignment operator (**=**) and an arithmetic operator
- It is used to change the value of a variable by adding, subtracting, multiplying, or dividing by a specified value
- The general form is

```
Variable Op = Expression
```

  which is equivalent to

```
Variable = Variable Op (Expression)
```

  - The **Expression** can be another variable, a constant, or a more complicated expression
  - Some examples of what **Op** can be are **+**, **-**, *****, **/**, or **%**

ASSG. STATEMENT

Shorthand Assignment Statement: Example

Example	Equivalent To
`count += 2;`	`count = count + 2;`
`sum -= discount;`	`sum = sum - discount;`
`bonus *= 2;`	`bonus = bonus * 2;`
`time /= rushFactor;`	`time = time / rushFactor;`
`change %= 100;`	`change = change % 100;`
`amount *= count1 + count2;`	`amount = amount * (count1 + count2);`

Arithmetic Operators and Expressions: Promotion

ASSG. STATEMENT

- If an arithmetic operator is combined with `int` operands, then the resulting type is `int`
- If an arithmetic operator is combined with one or two `double` operands, then the resulting type is `double`
- If **different types are combined in an expression**, then the resulting type is the right-most type on the following list that is found within the expression

```
byte → short → int → long → float → double
char ————————————↑
```

- Exception: If the type produced should be `byte` or `short` (according to the rules above), then the type produced will actually be an `int`

Precedence Rule

- Evaluate expressions in parentheses **separately** and the nested parentheses from the **inside out**.
- **Operator precedence rule**: Operators in the same expression are evaluated in the order **determined by their precedence** (from the highest to the lowest).
-

Operator	*Precedence*
method call	*highest precedence*
- (unary)	
`new`, type cast	
`*`, `/`, `%`	
`+`, `-` (binary)	
`=`	*lowest precedence*

- **Left associative rule**: Operators in the same expression and at the same precedence level are evaluated in **left-to-right order**.

Arithmetic Operators and Expressions: Parentheses

- An expression can be ***fully parenthesized*** in order to specify exactly what subexpressions are combined with each operator
- If some or all of the **parentheses in an expression are omitted**, **Java will follow *precedence* rules** to determine, in effect, where to place them
- However, it is best (and sometimes necessary) to include them

```
public class ParenthesesExample {
  public static void main(String[] args) { // Without parentheses: Java follows operator precedence
    int resultWithoutParentheses = 5 + 3 * 2; // Multiplication is performed first
    System.out.println("Result without parentheses: " + resultWithoutParentheses);

    // With parentheses: Change the order of evaluation
    int resultWithParentheses = (5 + 3) * 2; // Addition is performed first
    System.out.println("Result with parentheses: " + resultWithParentheses);
  }
}
```

Precedence Rule: Unary and Binary Operators

ASSG. STATEMENT

When two operations have **equal precedence**, the order of operations is determined by **associativity rules:**

- **Unary operators** of equal precedence are grouped **right-to-left**

  +-+rate is evaluated as **+(-(+rate))**

- **Binary operators** of equal precedence are grouped **left-to-right**:

  base + rate + hours is evaluated as

  (base + rate) + hours

- Exception: A string of assignment operators is grouped **right-to-left**

  n1 = n2 = n3; is evaluated as **n1 = (n2 = n3);**

Precedence Rule: Example

Evaluation of z – (a + b / 2) * w / y

```
z - (a + b / 2) * w / y   Operator, reason evaluated
        --/---             /, parentheses and precedence
    ---+--------             +, parentheses
    --------------*--        *, precedence, left associative
    -------------------/--    /, precedence
-- - ---------------------    -
```

Increment and Decrement Operator

Operator	Name	Description
++var	preincrement	The expression (++var) increments var by 1 and evaluates to the *new* value in var *after* the increment.
var++	postincrement	The expression (var++) evaluates to the *original* value in var and increments var by 1.
--var	predecrement	The expression (--var) decrements var by 1 and evaluates to the *new* value in var *after* the decrement.
var--	postdecrement	The expression (var--) evaluates to the *original* value in var and decrements var by 1.

Increment and Decrement Operator: Example

ASSG. STATEMENT

- If `n` is equal to `2`, then `2*(++n)` evaluates to `6`
- If `n` is equal to `2`, then `2*(n++)` evaluates to `5`
- Using increment and decrement operators **makes expressions short**, but it **also makes them complex** and difficult to read.
- **Avoid** using these operators in expressions that modify multiple variables, or the same variable for multiple times e.g.

```
int k = ++i + i;
```

The `String` Class

- Java has no primitive data type that holds a **series of characters**.
- The **String class from the Java standard library** is used for this purpose.
- In order to be useful, the a variable must be created to reference a `String` object.

```
String name;
```

- Notice the **S** in String **is upper case**.
- By convention, class names should always begin with an upper case character.

String Objects: Example

- A variable can be assigned a String literal e.g.
  `String ucapan1 = "Selamat";`
- Strings are the only objects that can be created in this way.
- A variable can be created using the new keyword:
  `String ucapan2 = new String("Sejahtera");`
- This is the method that all other objects must use when they are created.

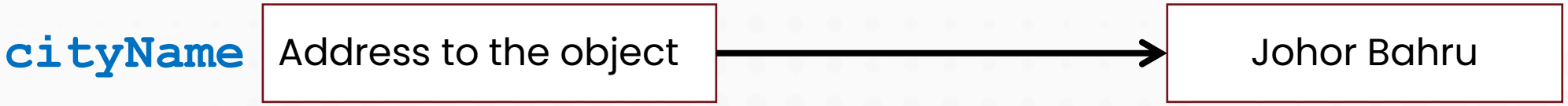
```
public class StringDemo {
 public static void main(String[] args) {
   String ucapan1 = "Selamat";
   String ucapan2 = new String("Sejahtera");
   System.out.print(ucapan1+" ");
   System.out.println(ucapan2);
 }
}
```

Reference Variables

- Objects are not stored in variables, however **objects are referenced by variables**.
- When a variable references an object, it contains the **memory address** of the object's location.
- Then it is said that the variable references the object.

```
String cityName = "Johor Bahru";
```

The object that contains the character string "Johor Bahru"



`String` Methods

- Since `String` is a class, objects that are instances of it have methods.
- One of those methods is the **`length`** method.

```
int stringSize;
stringSize = value.length();
```

- This statement runs the **`length`** method on the object pointed to by the **`value`** variable.
- The **`String`** class contains many methods that help with the manipulation of **`String`** objects.
- **`String`** objects are **immutable**, meaning that they cannot be changed and many of the methods of a **`String`** object can create new versions of the object.

String Method: length()


```
public class StringDemo1 {
 public static void main(String[] args) {
   String greeting = “Welcome”;
   String name = “Anne!”;
   int size;
   size = greeting.length();
   System.out.println(“Length of “+greeting+ “ is: “+size);
   size = name.length();
   System.out.println(“Length of “+name+ “ is: " +size);
 }
}
```

String Methods: `toLowerCase()`, `toUpperCase()`, `charAt()`

```
public class StringDemo2 {
 public static void main(String[] args) {
   String location = "UTM Skudai";
   String upper = location.toUpperCase();
   String lower = location.toLowerCase();
   char letter = location.charAt(2);
   int size = location.length();

   System.out.println(location);
   System.out.println(upper);
   System.out.println(lower);
   System.out.println(letter);
   System.out.println(size);
 }
}
```

The `Scanner` Class

- To read **input from the keyboard** we can use the **`Scanner`** class.
- The **`Scanner`** class is defined in **`java.util`**, so we will use the following statement at the top of our programs:

```
import java.util.Scanner;
```

- Scanner objects work with **`System.in`**
  - To create a **`Scanner`** object:

```
Scanner keyboard = new Scanner (System.in);
```

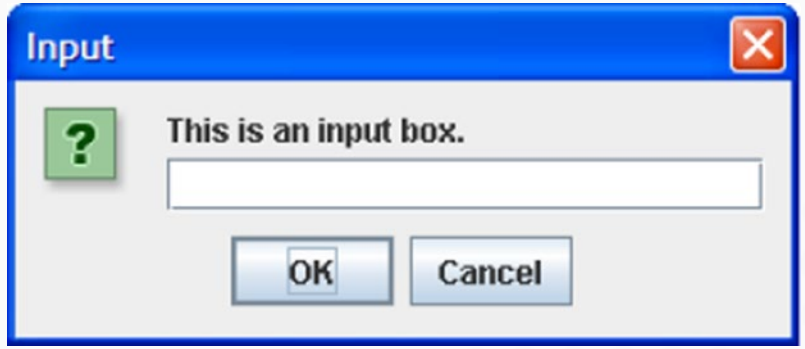
- Based on Java API, we cam list the Scanner class methods.

The `JOptionPane` Class

- The **`JOptionPane`** class is not automatically available in our Java programs.
- The following statement must be in the program's class header:

```
import javax.swing.JOptionPane;
```

- This statement tells the compiler where to find the **`JOptionPane`** class.
- The **`JOptionPane`** class provides methods to display each type of **dialog box**.



Message Dialogs: Example

- `JOptionPane.showMessageDialog` method is used to display a message dialog.

```
JOptionPane.showMessageDialog(null, "Hello World");
```

- The first argument will be discussed in Chapter 7
- The second argument is the message that is to be displayed.

Input Dialogs

- An input dialog is a quick and simple way to **ask the user to enter data**.
- The dialog displays a text field, an OK button and a Cancel button.
- If button **OK is pressed**, the dialog **returns the user's input**.
- If button **Cancel is pressed**, the dialog **returns null**.

Input Dialogs: Example

- The **argument passed** to the method is the message to display.
- If the user clicks on the OK button, **`name`** references the string entered by the user.
- If the user clicks on the Cancel button, **`name`** references **`null`**.

```
String name;
Name = JOptionPane.showInputDialog("Enter your name.");
```

`System.exit` Method

- A program that uses **`JOptionPane`** does not automatically stop executing when the end of the main method is reached.
- Java generates a thread, which is a process running in the computer, when a **`JOptionPane`** is created.
- If the **`System.exit`** method is not called, this thread continues to execute.

`System.exit` Method

- The **`System.exit`** method requires an integer argument:

```
System.exit(0);
```

- This argument is an **exit code** that is passed back to the operating system.
- This code is usually ignored. However, it **can be used outside the program**:
  - To indicate whether the program ended successfully or as the result of a failure
  - The value 0 traditionally indicates that the program ended successfully

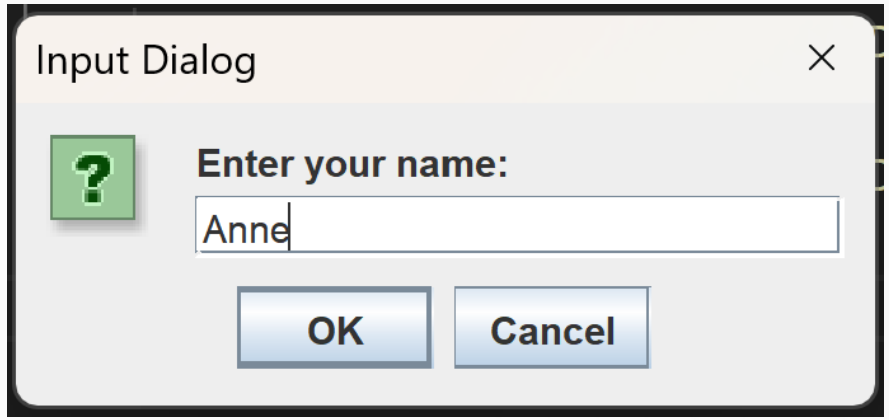
showInputDialog and showMessageDialog: Example

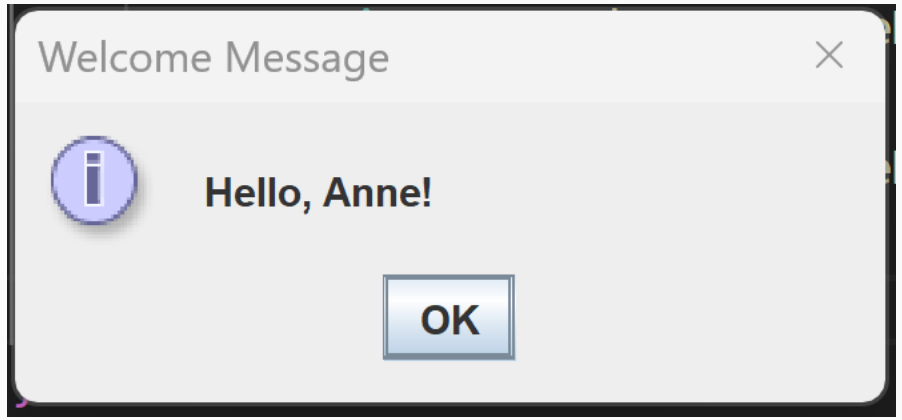

```
import javax.swing.JOptionPane;

public class InputDialogExample {
    public static void main(String[] args) {
       // Show input dialog and capture user input String userInput =
       JOptionPane.showInputDialog(null, "Enter your name:", "Input Dialog",
       JOptionPane.QUESTION_MESSAGE);

       // Display the entered name using a message dialog
       if (userInput != null && !userInput.isEmpty()) {
            JOptionPane.showMessageDialog(null, "Hello, " + userInput + "!",
"Welcome Message", JOptionPane.INFORMATION_MESSAGE);
        } else {
            JOptionPane.showMessageDialog(null, "No name entered!",
"Warning", JOptionPane.WARNING_MESSAGE);
        }
    }
}
```

`showInputDialog` and `showMessageDialog`: Example





Converting a String to a Number

- The **`JOptionPane`**'s **`showInputDialog`** method always returns the user's input as a **`String`**
- A **`String`** containing a number, such as "42" can be converted to a **numeric data type**.
- E.g. convert a `String` to an `int`

```
public class StringToIntExample {
    public static void main(String[] args) {
        String numberString = "42"; // String containing a number
        int number = Integer.parseInt(numberString); // Convert to int
        System.out.println("The integer value is: " + number);
    }
}
```

Parse Method

- Each of the **numeric wrapper classes**, (covered in Chapter 10) has a method that converts a string to a number.
  - The **`Integer` class** has a method that converts a **`string`** to an **`int`**
  - The **`Double` class** has a method that converts a string to a **`double`**, and
  - Others
- These methods are known as ***parse* methods** because their names begin with the word "parse."

```
// Store 1 in bVar.
byte bVar = Byte.parseByte("1");

// Store 2599 in iVar.
int iVar = Integer.parseInt("2599");

// Store 10 in sVar.
short sVar = Short.parseShort("10");

// Store 15908 in lVar.
long lVar = Long.parseLong("15908");

// Store 12.3 in fVar.
float fVar = Float.parseFloat("12.3");

// Store 7945.6 in dVar.
double dVar =
Double.parseDouble("7945.6");
```

Reading an `Integer` and a `Double` with an Input Dialog

```
int number;
String str;
str = JOptionPane.showInputDialog("Enter a number:");
number = Integer.parseInt(str);
```

```
double price;
String str;
str = JOptionPane.showInputDialog("Enter the retail price.");
price = Double.parseDouble(str);
```